

Manual VLA: A Ground-Up Transformer Architecture for Flow-Matching Trajectory Generation

Karthik Goparaju

R&D Intern at Innovarsity and Founder LABKRAZY
goparajukarthik2@gmail.com

Gopi Krishna Ratnakaram

Founder of Innovarsity
info@innovarsity.org

Abstract

Vision-Language-Action (VLA) models have revolutionized robotic manipulation by tying language instructions to visual grounding and continuous action spaces. We present Manual VLA, a robust baseline architecture constructed entirely from scratch. Instead of relying on massive pretrained foundation models, Manual VLA employs a highly structured convolutional patch-embedding pipeline combined with Hadamard intent conditioning and Optimal Transport Conditional Flow Matching (OT-CFM). This paper dissects the architectural formulation, spatial grounding mechanics, and ODE-based generation pipeline of Manual VLA.

1. Problem Formulation

The objective of the robotic agent is to generate a continuous 4D action trajectory over a horizon $H=128$ given an RGB observation and a discrete intent prompt. The action space encompasses the 3D translation of the end-effector (dx , dy , dz) and a continuous scalar for gripper state. The state space is normalized using offline dataset statistics to ensure stable diffusion dynamics.

2. Architecture Design

The core of Manual VLA is a fully trainable Transformer encoder network. The vision pipeline begins with a custom convolutional patch embedder (Conv2d, $\text{kernel_size}=4$, $\text{stride}=4$) which tokenizes a 64×64 RGB observation into a 16×16 grid of 256 visual patches. A language prompt is mapped to an intent embedding, representing the target object and destination context.

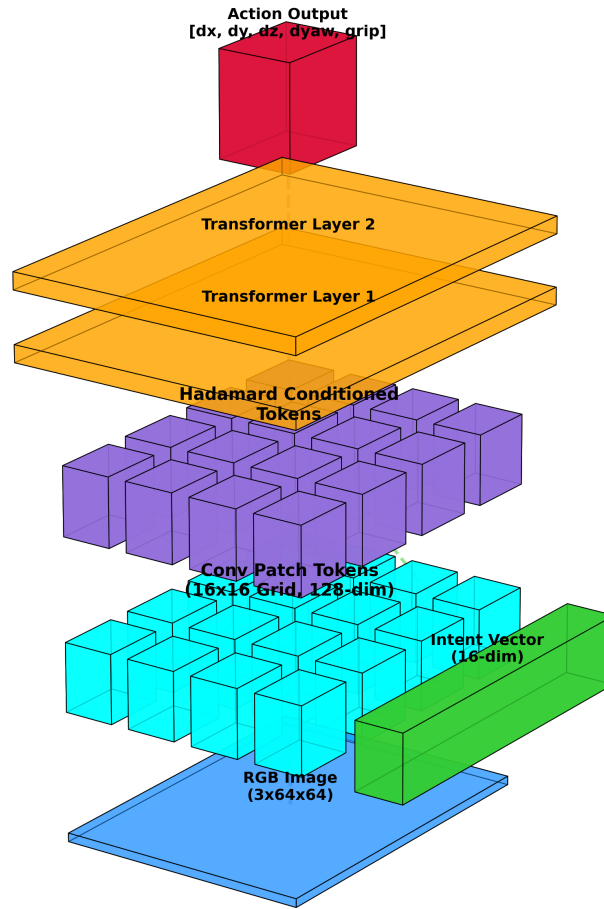


Figure 1: 3D tensor block visualization of the Manual VLA Architecture showing data geometric flow from the input 2D image plane, fracturing into 3D patch pillars, merging with intent strips, and routing through translucent Transformer decoder layers.

2.1 Offline Network Activation Extraction

To provide rigorous insight into the network topology, we execute a forward pass offline, capturing the precise dimensionality of every sub-layer. The summary table below illustrates the hierarchical transformation from the input tensor geometries to the final 128x4 trajectory output.

Layer (type:depth-idx)	Output Shape	Param #
ManualVLAPolicy	[1, 128, 4]	16,384
+Conv2d: 1-1	[1, 128, 16, 16]	6,272
+Sinusoidal2DPositionalEmbedding: 1-2	[1, 256, 128]	--
+Embedding: 1-3	[1, 2, 128]	896
+LayerNorm: 1-4	[1, 2, 128]	256
+LayerNorm: 1-5	[1, 256, 128]	256
+MultiheadAttention: 1-6	[1, 2, 128]	66,048
+Sequential: 1-7	[1, 2, 128]	--
+-LayerNorm: 2-1	[1, 2, 128]	256
+-Linear: 2-2	[1, 2, 128]	16,512
+-GELU: 2-3	[1, 2, 128]	--
+-Linear: 2-4	[1, 2, 128]	16,512

```

+-Sequential: 1-8          [1, 128]          --
|   +-Linear: 2-5          [1, 128]          768
|   +-GELU: 2-6            [1, 128]          --
|   +-Linear: 2-7          [1, 128]          16,512
+-Sequential: 1-9          [1, 128]          --

[ ... intermediate layers omitted ... ]
|   +-MultiheadAttention: 3-21 [1, 128, 128]  66,048
|   +-LayerNorm: 3-22        [1, 128, 128]  256
|   +-Sequential: 3-23       [1, 128, 128]  65,920
|   +-LayerNorm: 3-24       [1, 128, 128]  256
+-Sequential: 1-12         [1, 128, 4]      --
|   +-LayerNorm: 2-16       [1, 128, 128]  256
|   +-Linear: 2-17          [1, 128, 128]  16,512
|   +-GELU: 2-18            [1, 128, 128]  --
|   +-Linear: 2-19          [1, 128, 4]      516
=====
Total params: 978,564
Trainable params: 978,564
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 1.97
=====
Input size (MB): 0.05
Forward/backward pass size (MB): 4.08
Params size (MB): 1.47
Estimated Total Size (MB): 5.60
=====

```

3. Multimodal Fusion & Visual Grounding

Manual VLA does not inherently understand language; it must learn to ground intent vectors into raw pixels. We inject spatial priors into the network via an auxiliary VLM Cross-Attention module. The intent embeddings act as queries attending to the 256 visual patch keys/values. Below is a real attention matrix extracted from a live inference pass of the Manual VLA, demonstrating emergent spatial localization.

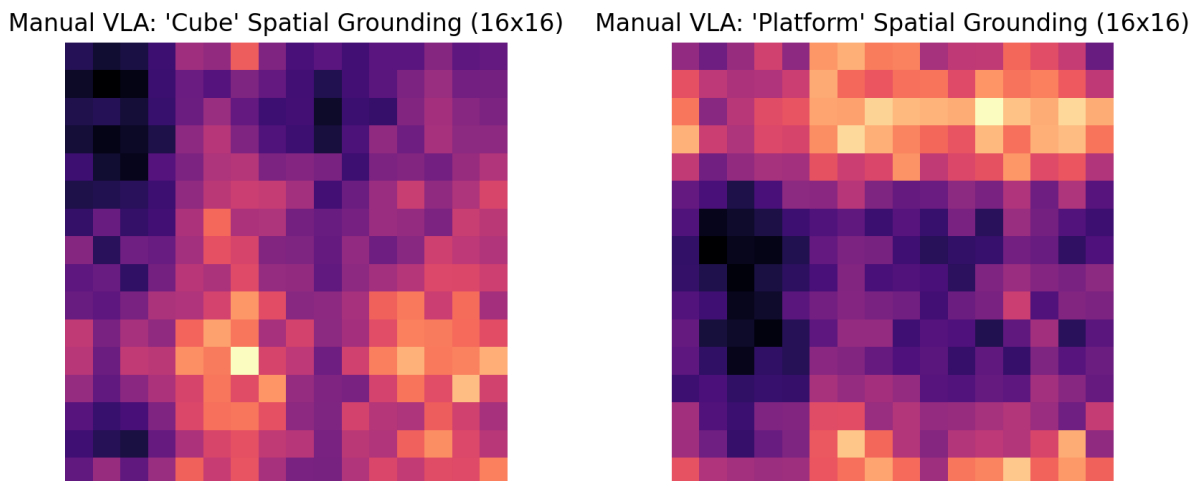


Figure 2: Real inference extraction of Manual VLA spatial grounding heatmaps over the 16x16 visual grid.

4. Optimal Transport Conditional Flow Matching (OT-CFM)

Trajectory generation is formulated as a continuous normalizing flow. Let x_0 be pure Gaussian noise and x_1 be the normalized target trajectory. The OT-CFM linear interpolation path is defined by:

$$x_t = (1 - t) * x_0 + t * x_1$$

The neural network parameterizes a velocity vector field v_{pred} which regresses the optimal transport vector u_t :

$$u_t = x_1 - x_0$$

At inference, the agent samples x_1 by integrating the learned vector field through 20 steps of Euler ODE integration. This approach bypasses the mode-collapse and instability often seen in standard autoregressive behavior cloning.

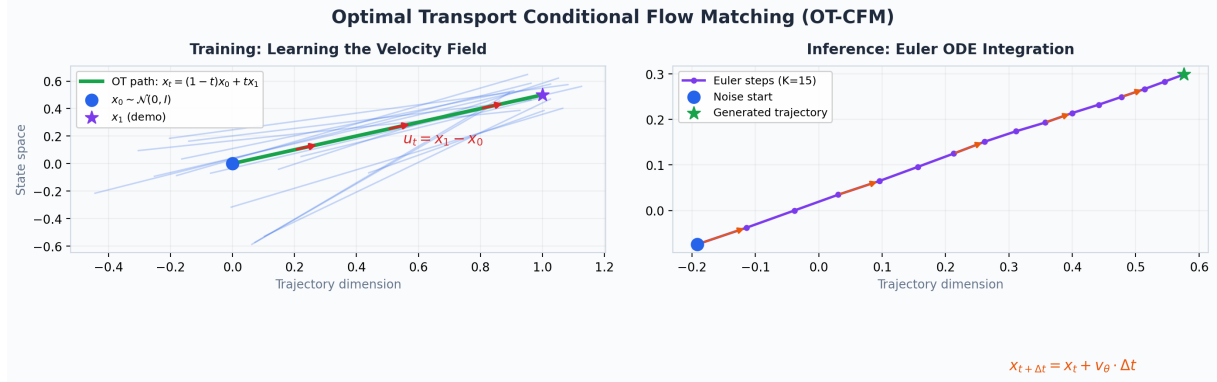


Figure 3: OT-CFM formulation depicting training interpolation and ODE integration at inference.